IT3091- Machine Learning

Assignment - Guided Data Track

DataCo Smart Supply Chain: Late-Delivery Risk

Final Group Report

Group Number: 2026-AI-05

Date of Submission: 12 October 2026

Table of Contents

1. Group members
2. Executive summary
3. Problem framing
4. Workflow
5. Data understanding and EDA
6. Preprocessing and feature engineering
7. Models and comparison
8. Evaluation and validation
9. Recommendation
10. Limitations and risks
11. Decision log
12. Reproducibility
13. AI-use declaration
14. Member contributions
15. Appendix

1. Group members

Member	IT Number	Focus
Bolonne B.R.M.	IT24102050	EDA + preprocessing
Kodagoda D.S.	IT24101613	EDA + baseline model training
Jayasekara C.D.	IT24103494	EDA + hyper parameter tuning
Liyanage J.L.K.L.	IT24101927	EDA + Evaluation and recommendation

2. Executive summary

DataCo's late deliveries are caused by the delivery promise, not by the delivery network. We found this while building a model to predict late-delivery risk, and it turned out to matter more than the model.

First Class orders are late 100% of the time, all 16,948 of them in our training data, across all thirteen quarters. Every First Class shipment takes exactly two days against a one-day promise, so the tier cannot succeed. More tellingly, Second Class and Standard Class have identical actual transit times (2-6 days, median 4), yet are late 80% and 40% respectively. The only difference between them is the number printed on the tier.

We trained a baseline and four alternative models on 172,765 shipped order line items. Our best model, a tuned Random Forest, reached ROC-AUC 0.7807 on a held-out test set covering May 2017 to January 2018. The Logistic Regression baseline reached 0.7471. For comparison, a four-column rule using only Shipping Mode reaches 0.7387, so the machine learning adds about 0.03 over a lookup table.

We recommend DataCo re-baseline its expedited SLAs, First Class should promise two days, not one, and publicise its midday dispatch cut-off, since orders placed after noon are late 6.4 percentage points more often. The model is the evidence for these changes rather than the product itself.

3. Problem framing

Stakeholder: DataCo's logistics operations team, who decide how an order is shipped and whether to intervene on it.

Decision need: They want to know, at the moment an order is placed, whether it is at risk of arriving late, early enough to do something about it, such as upgrading the shipping mode, warning the customer, or re-routing the shipment. A prediction that arrives after dispatch is useless to them.

Primary lens: late-delivery risk. We chose this because the dataset already contains a clean 0/1 target, Late_delivery_risk, so the label is well defined from day one rather than invented by us. It turns naturally into a binary classification problem, which lets us run a proper baseline-plus-alternatives comparison with real evaluation metrics. Most importantly, it answers a question the business actually has.

Secondary lens: delay pattern analysis. We did not build a separate pipeline for this. We used it as the explanatory layer on top of the primary lens: once the model could rank risk, we looked at feature importance and EDA breakdowns to explain what drives lateness. This kept the two lenses connected and

made our final recommendation specific rather than generic. We deliberately did not take a third lens, since shipment priority profiling overlaps heavily with late-delivery risk and would have added work without adding coherence.

Unit of analysis: One order line item, one row in the dataset. This matters: 180,519 rows describe only 65,752 distinct orders, roughly 2.75 line items each, and all items of one order share a single delivery outcome. That fact drove our splitting strategy.

Task and output: Supervised binary classification. For each order, using only information known at order time, output a probability that the order will be delivered late, plus a yes/no flag at a chosen decision threshold.

Scope decision: We excluded the unstructured clickstream file (tokenized_access_logs.csv). It contains no Customer Id or Order Id, so it cannot be joined to order records, and browsing behaviour does not inform whether a shipment arrives on time. It answers a different business question.

4. Workflow

Our workflow runs in six steps from the business problem to the recommendation. Each step feeds the next, and the decisions made at each one are recorded in the decision log.

Step	What we did	What it produced
1. Data understanding	Read every column against the data dictionary, noted types, and marked leakage risks	Column audit table
2. EDA	Checked class balance, delay rates by shipping mode, market, region and category, and looked for missing values, duplicates and outliers	31 figures and the EDA findings
3. Preprocessing	Dropped leakage and non-predictive columns, removed cancelled orders, split the data, encoded and scaled	Clean train/validation/test splits and a fitted pipeline
4. Modelling	Trained a baseline first, then four alternatives	Seven rows of results
5. Evaluation	Compared on precision, recall, F1 and ROC-AUC, cross-validated, and re-checked for leakage	Model comparison and final test score
6. Recommendation	Tied the results back to the secondary lens	Evidence-based recommendation with

Step	What we did	What it produced
		limitations

As a flow:

Business problem → Data understanding → EDA → Preprocessing → Modelling → Evaluation → Recommendation

Two steps loop back. Findings in EDA changed what we dropped in preprocessing, and the leakage check in evaluation sent us back to re-examine the cross-validation splitter.

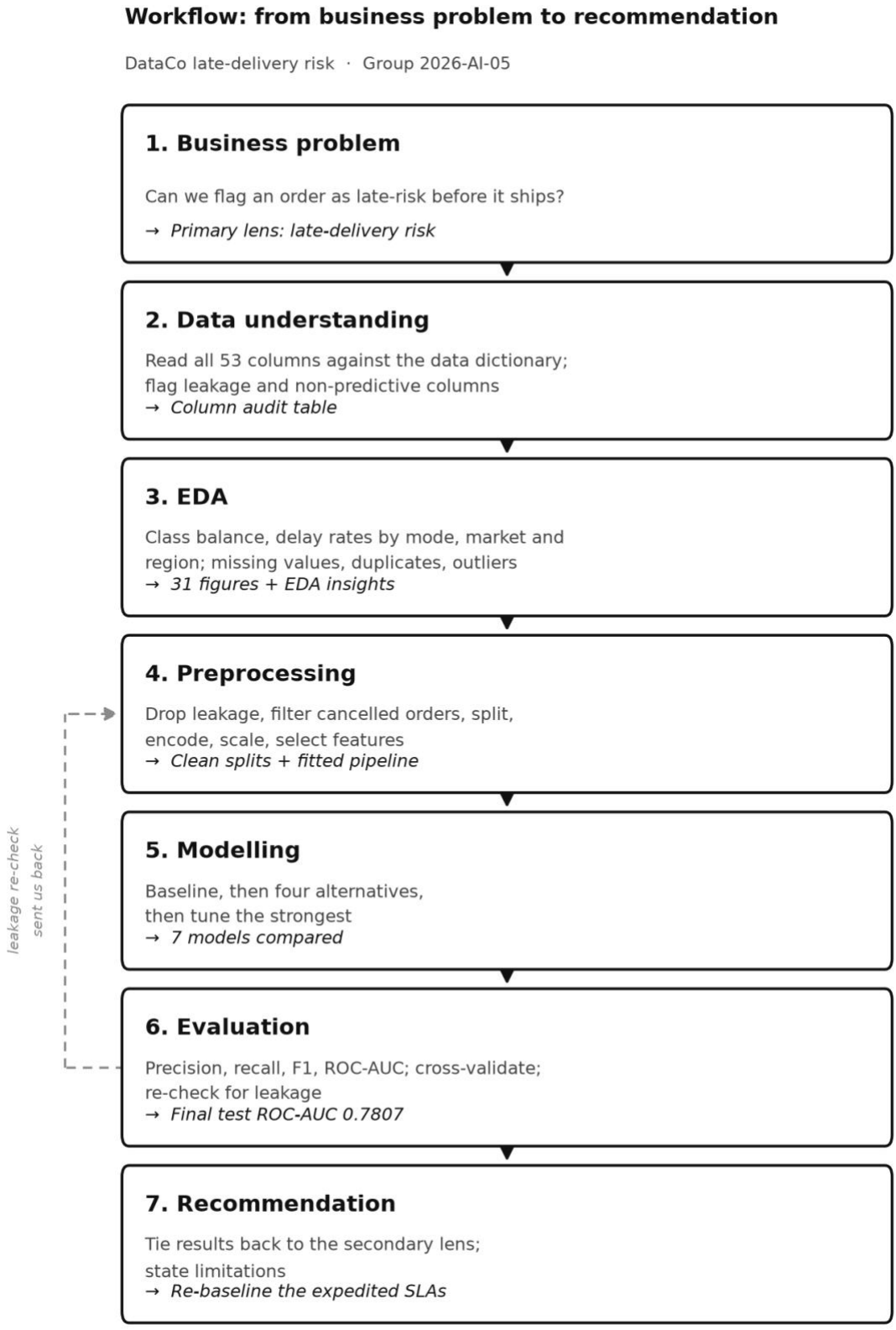


Figure 1: Workflow Diagram

5. Data understanding and EDA

The DataCo Smart Supply Chain dataset holds 180,519 rows and 53 columns covering January 2015 to January 2018. One row is one order line item, not one order.

Data quality

The file is Latin-1 encoded, not UTF-8. Reading it as UTF-8 corrupts Spanish place names such as Rajastán, which would split one real category into two during encoding, so we set the encoding explicitly.

Issue	Finding	What we did
Duplicate rows	none; Order Item Id is a clean key	nothing needed
Missing values	only 4 columns affected	all 4 dropped for independent reasons
Product Description	100% null	dropped
Order Zipcode	86% missing, in structured bands	dropped, destination already in Order Region
Constant columns	Customer Email, Customer Password, Product Status	dropped
Duplicate columns	6 exact pairs found by element-wise comparison	kept one of each
Customer State	2 junk values (zip codes in a state field, 3 rows)	handled by the encoder

After the column drops, zero missing values remain in the feature set.

What drives lateness

Shipping Mode dominates everything else. Cramér's V against the target is 0.48 for Shipping Mode and below 0.03 for every other categorical feature. Customer Segment, Customer Country and Department Name score effectively 0.00.

Mode	Promised	Actual days (min-median-max)	Late rate
First Class	1	2 - 2 - 2	100.0%
Second Class	2	2 - 4 - 6	79.9%
Same Day	0	0 - 0 - 1	45.9%
Standard Class	4	2 - 4 - 6	39.9%

The second and fourth rows are the finding. Second Class and Standard Class have the same actual transit distribution, but one promises two days and the other four, so one fails 80% of the time and the other 40%. The network behaves consistently; the promises do not.

What does not drive lateness

- **Geography:** Market late rates sit between 55.0% and 58.0% across five continents. The mode effect is identical in every market and in every customer segment, so there is no interaction to exploit.
- **Order value:** Median price, line total and quantity are the same across all four shipping modes, which rules out the objection that expedited shipping is simply chosen for expensive orders.
- **Numeric features generally:** Late and on-time distributions overlap almost exactly. The largest standardised mean difference across all 17 numeric features is 0.104 (order_hour), below the 0.2 threshold normally called negligible.

The one secondary signal

Hour of day is the exception. The late rate climbs from 51.7% for orders placed at 11:00 to 62.0% at 20:00, and the pattern is a step rather than a smooth trend: 54.2% before noon, 60.6% after. That is the signature of a midday dispatch cut-off, orders placed after it miss the day's outbound van and lose a day at the depot.

Stability over time

The monthly late rate is flat across all 37 months, with a standard deviation of 0.01 and no trend or seasonality. There is no concept drift in the target, which is what made a chronological split safe.

6. Preprocessing and feature engineering

Data Leakage

We suspected the target was simply derived from the data, so we tested it rather than assuming. The rule "real shipping days > scheduled days" reproduces the label on 97.55% of rows. It is the label.

We dropped four columns on that basis:

Column	Why
Days for shipping (real)	actual shipping duration; the label is arithmetic on it
Delivery Status	the same outcome written out in words
shipping date (DateOrders)	actual dispatch timestamp, known only afterwards
Order Status	a lifecycle state that evolves *after* the order is placed

The last one was not in our initial plan. We added it because it fails the same test: at prediction time every order is effectively "just placed", so a status of COMPLETE is future information.

Had we left these in, every model would have scored near 100% for the wrong reason.

Scope, removing orders that never shipped

Every row where the arithmetic rule disagreed with the label turned out to be a cancelled shipment. Those orders never shipped, so "late" is undefined for them and a label of 0 does not mean "on time". We removed the 7,754 cancelled and suspected-fraud rows (4.3% of the data) after verifying the two groups coincide exactly.

This raised the class balance from 54.8% to 57.3% late, which is the honest figure: among orders that actually ship, well over half arrive late.

The split, why not random

We used a chronological split, not a random one, for two independent reasons.

1. Rows are not independent. One order spans up to five rows that share a single label. A random row split would put item 1 of an order in training and item 3 in test, answer included.
2. The real task is forward in time. DataCo would train on history and score orders that have not happened yet. A random split lets a model train on 2017 and test on 2016, which is an easier problem than the real one.

A date split closes both at once, because an order has exactly one order date.

Split	Period	Rows	Late rate
Train	2015-01-01 → 2016-10-31	109,871	57.36%
Validation	2016-11-01 → 2017-04-30	29,842	57.09%
Test	2017-05-01 → 2018-01-31	33,052	57.25%

We asserted in code that no Order Id appears in two splits and that the three periods are strictly ordered. Every chart in our EDA was drawn on the training set only, because a choice made after looking at a chart is a choice fitted to that data.

Column decisions

All 53 original columns were assigned to exactly one bucket with a written reason, and the notebook asserts that the buckets partition the column list so nothing can be silently forgotten. 35 columns were dropped: leakage (4), identifiers (9), redundant duplicates (6), no information (5), high-cardinality geography (4), personal data (4), post-hoc financials (2), sparse (1).

One redundancy is worth singling out. Order Item Total equals Product Price × Order Item Quantity minus Order Item Discount, and this holds on 100.00% of rows, yet Pearson correlation never exceeds 0.91 between any pair, because the relationship is multiplicative. A correlation matrix alone will not catch that class of redundancy.

Feature engineering

We decomposed the order timestamp into ten calendar features. The one piece of real engineering is is_afternoon, a binary flag for orders placed from 12:00 onward. Section 4 showed the late rate is a step at noon, not a trend, so a raw hour number fits a straight line through a step and captures only part of it. Adding the flag moved validation ROC-AUC from 0.742 to 0.747.

We deliberately excluded order_year. Training covers 2015-16 and test covers 2017-18, so the year is always outside the training range at prediction time. That is extrapolation, not learning.

The pipeline

Preprocessing is a scikit-learn ColumnTransformer fitted on the training set only, so it cannot leak, and it sits inside the model pipeline so it is re-fitted on every cross-validation fold.

Step	Choice	Why
Numeric missing	median imputation	robust to the right-skew in monetary columns
Numeric scaling	StandardScaler	required by Logistic Regression; harmless to trees
Categorical missing	constant "Missing"	makes absence an explicit, learnable category
Encoding	OneHotEncoder	all 9 categoricals are nominal, with no natural order
Unseen categories	handle_unknown = infrequent_if_exist	essential under a temporal split, see below
Outliers	kept	verified as genuine high-value orders, not errors
Class imbalance	no resampling	1.35:1 is mild; handled by metric and threshold choice

26 raw features expand to 132 encoded columns.

The unseen-category problem is real here, not hypothetical. Five departments and 26 product categories launched in 2017, so they appear only in the test period, touching roughly 13,690 rows. With the default handle_unknown setting of "error" the notebook would have crashed at the final evaluation. A random split would have scattered these across all three sets and hidden the problem entirely.

Feature selection

Selection happened at two levels. The first, the 35 columns above, was manual, because no algorithm could discover that Delivery Status must go: an automatic selector would happily keep it precisely because it predicts perfectly.

The second level was statistical: variance threshold, pairwise correlation, mutual information, L1 coefficients and Random Forest importance, rank-averaged. We then tested whether pruning actually helps.

Feature set	Columns	Validation ROC-AUC
All encoded features	132	0.7471
Drop redundant (correlation > 0.95)	128	0.7473
Top 25 by mean rank	25	0.7521
Shipping Mode dummies only	4	0.7387

Four one-hot columns reach within 0.013 of all 132. We kept the full set, since the spread across the first three is smaller than the standard error and tree models perform their own selection anyway.

One methodological note worth recording: mutual information ranked Latitude second, above three shipping modes, despite it having a correlation of about 0.00 with the target. This is a known bias in the k-nearest-neighbour estimator for continuous features with many distinct values. Both the L1 model and the Random Forest gave it near-zero importance. Relying on a single filter would have produced a wrong conclusion.

Why we did not use PCA

We tested it rather than assuming. At equal dimensionality, feature selection beat PCA by 0.020 ROC-AUC (25 selected features scored 0.7521 against 0.7323 for 25 principal components), and PCA needed 50 components just to match the full model. At two components it scored 0.4954, which is worse than guessing.

The reasons are structural: with 132 features and 109,871 rows there is no dimensionality problem to solve; 115 of the 132 columns are one-hot binaries, which PCA handles poorly; and PCA is unsupervised, so it optimises variance rather than relevance. The Same Day shipping dummy is only 5% of rows and therefore carries little variance, yet it matters. Finally, PCA would have destroyed the deliverable, our report has to name which factors drive lateness, not describe a mixture of 132 columns.

7. Models and comparison

We trained one baseline and four alternatives. We also added two reference points that are not really models, so that every score below can be read against something.

Why each model is here

Model	Type	The question it answers
Majority class	-	What does doing nothing score?
Shipping Mode rule	-	Can a simple lookup table do the job?
Logistic Regression	linear	Baseline. How far does a simple model get?
Decision Tree	one tree	Does non-linearity help at all?
Random Forest	many trees averaged	Does averaging trees reduce error?
XGBoost	trees built in sequence	Does error-correction beat averaging?
Neural network (MLP)	neural	Does more model power help?

We chose Logistic Regression as the baseline because it is simple, fast, and easy to read, its coefficients tell you which factors raise or lower risk. Our EDA also showed no interaction between shipping mode and destination, so a simple additive model is not unfairly handicapped here. That makes it a fair baseline rather than an easy target.

We rejected two common models for specific reasons. KNN measures distance between rows, but 115 of our 132 columns are 0/1 dummies, so distance means very little, and it is slow at this size. Naive Bayes assumes features are independent, which one-hot columns of the same feature break by definition.

Results

All scores are on the validation set. ROC-AUC is our headline metric.

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Random Forest (tuned)	0.7147	0.8672	0.5908	0.7028	0.7704
Random Forest	0.7194	0.8917	0.5789	0.7020	0.7694
XGBoost	0.7038	0.8089	0.6299	0.7083	0.7676
Decision Tree	0.7092	0.8538	0.5920	0.6992	0.7673

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Logistic Regression (baseline)	0.7058	0.8813	0.5601	0.6849	0.7471
Neural network (MLP)	0.6612	0.7153	0.6754	0.6948	0.7407
Shipping Mode rule	0.6944	0.8844	0.5346	0.6664	0.7387
Majority class	0.5709	0.5709	1.0000	0.7268	0.5000

Three things stand out.

1. Tree models beat the baseline, but only by about 2 points. Random Forest gains 0.022 ROC-AUC over Logistic Regression.
2. The neural network did worse than the baseline. More model power did not help, which supports our view that the limit is in the data, not the method.
3. The Shipping Mode rule gets 0.7387 using four columns. Our best model gets 0.7704. So all the machine learning adds about 0.03 over a rule you could write by hand.

Hyperparameter tuning

We tuned the Random Forest using a randomised search with 12 candidate settings, scored on ROC-AUC, using the same cross-validation as everything else.

Best settings: 200 trees, maximum depth 20, minimum 10 samples per leaf, and half the features considered at each split.

Model	Cross-validated ROC-AUC
Untuned Random Forest	0.7747
Tuned Random Forest	0.7760
Gain from tuning	+0.0012
Fold-to-fold variation	0.0023

The gain is smaller than the model's own run-to-run variation. So the tuned model is different, not clearly better. We report it for completeness but do not claim it as a real improvement. This is consistent with everything else we found: the signal is concentrated in one feature, so tuning cannot move the result much.

We tuned only the strongest model, as planned. Since a full search on the best model could not escape normal variation, there is no reason to think tuning the others would change the ranking.

8. Evaluation and validation

Why we do not report accuracy alone

57% of shipped orders are late. A model that always says "late" is therefore 57% accurate while knowing nothing at all, its ROC-AUC is exactly 0.50. Accuracy on its own would make a useless model look reasonable, so we report precision, recall, F1, ROC-AUC and PR-AUC alongside it.

Cross-validation

We used a time-series split with 5 folds, which trains on earlier data and validates on later data. That matches how the model would really be used.

We then found a problem with it. TimeSeriesSplit cuts the data at a row position, and the rows of one order sit next to each other. So a fold boundary can land in the middle of an order, putting some of its items in training and the rest in validation, with the answer attached. We measured it: 2 orders, 4 rows. Small, but it is exactly the leak we avoided a random split for.

We wrote a small custom splitter that drops any validation row whose order already appeared in training. The notebook now asserts that 0 orders straddle a fold.

Are the models actually different?

This is the question we think matters most, and it is easy to skip.

Model	Cross-validated ROC-AUC
Random Forest	0.7746 ± 0.0023
XGBoost	0.7741 ± 0.0032
Decision Tree	0.7724 ± 0.0034

The gap between the top three is 0.0022. The normal variation between folds of a single model is about 0.0030. The difference between models is smaller than the wobble within one model, so the top three are statistically tied.

We therefore do not claim Random Forest is the best model. We chose it because it was the most stable of the three (smallest fold-to-fold variation), gives readable feature importances, and needs no extra library.

Choosing a decision threshold

Every model above used the default cut-off of 0.5. That default assumes a false alarm and a missed late order cost the same. For DataCo they do not: missing a late order can lose a customer, while a false alarm costs one cheap shipping upgrade. So recall matters more than precision here.

We found a trap while checking this. Maximising F1 picks a threshold that flags 97% of all orders, which is the "always say late" model wearing a disguise. F1 is not a safe thing to optimise on this data.

The reason is that predicted probabilities form a few separate clusters rather than a smooth spread, because Shipping Mode dominates. An order is basically either expedited (high probability) or standard (low). So we are not turning a dial, we are choosing between a few fixed operating points.

Threshold	Orders flagged	Recall	Precision
0.50 (default)	39%	0.598	0.883
0.40 (higher recall)	66%	0.788	0.686

In plain terms, out of 18,922 genuinely late orders in the test set: at 0.50 we catch 11,315 and miss 7,607. At 0.40 we catch 14,905 and miss 4,017.

Final test result

We opened the test set once, at the very end, after all model choices were made.

Metric	Validation	Test
Accuracy	0.7147	0.7244
Precision	0.8672	0.8828
Recall	0.5908	0.5980
F1	0.7028	0.7130
ROC-AUC	0.7704	0.7807

The test score is slightly higher than validation, by 0.0103. We are not treating that as proof of a better model, it just means the test window happened to be slightly easier. It is worth noting that about 13,690 test rows contain product categories that did not exist during training, since DataCo's catalogue expanded in 2017, which would normally push the score down.

Checking the model against the EDA

The tree models rank Shipping Mode at the top, which is what our EDA predicted before any model was trained. Two independent methods reaching the same conclusion is stronger evidence than either on its own, and it links the model back to our secondary lens.

9. Recommendation

Our main recommendation is not "deploy this model". It is to fix the delivery promises, and the model is the evidence for why.

1. Re-baseline the expedited shipping tiers

First Class promises one day and delivers in exactly two, every single time. It cannot meet its promise, so it fails 100% of the time by definition rather than by accident.

The clearest evidence is the comparison between Second Class and Standard Class. They have the same actual transit times, 2 to 6 days, median 4, but one promises 2 days and the other 4. As a result one is late 80% of the time and the other 40%. Nothing about the operation differs; only the promise does.

What to do: change First Class to a 2-day promise and review Second Class against its real 4-day median. On the training data this alone would move First Class from 100% late to roughly 0% late, with no operational change at all.

This also works everywhere at once. The shipping-mode effect is identical across all five markets and all three customer segments, so one global change fixes it rather than needing regional work.

2. Publicise the midday dispatch cut-off

Orders placed from 12:00 onward are late 60.6% of the time, against 54.2% before noon, a 6.4 point difference, with a smooth rise through the day to 62.0% at 20:00.

What to do: show customers the cut-off time at checkout, and consider a second afternoon dispatch for high-value or expedited orders. This is a smaller effect than the SLA problem, but it is a real and fixable mechanism.

3. Use the model for triage, not for the headline

If DataCo wants to use the model, we suggest running it at a threshold of 0.40 rather than the default 0.50. That catches 79% of late orders instead of 60%, at the cost of flagging 66% of orders rather than 39%.

We would be honest with the business about what this is worth. The model adds about 0.03 ROC-AUC over a rule that uses only the shipping mode. Most of its value is confirming where the problem is, not sorting individual orders.

Expected value

We cannot put a currency figure on this, because the dataset has no cost data, no penalty clauses, refund amounts or carrier charges. What we can say is that in the test period, 18,922 of 33,052 shipped orders arrived late, and the single largest identified cause is a promise the operation was never able to meet.

10. Limitations and risks

The data has no operational detail. There is no carrier, warehouse, route, weather, customs or capacity information. Those almost certainly explain why two orders on the same shipping mode can differ, and we cannot model any of it. The ceiling of about 0.78 ROC-AUC is a property of the dataset, not of the methods we chose. A better model will not fix this; more columns would.

The target is calculated, not observed. Late_delivery_risk is derived arithmetically from the recorded shipping dates, so it inherits any error in those dates. If a dispatch date was logged wrongly, the label is wrong too, and we have no way to detect that.

The timestamps look artificial. Order volume is almost perfectly uniform across days of the week, with no weekend dip. Real retail almost always shows one. This suggests the order dates may be synthetic or evenly resampled, which weakens any conclusion that depends on timing, including our midday cut-off finding, which should be verified against DataCo's real dispatch records before acting on it.

Geographic scope is narrow. All store locations are in Puerto Rico or the USA, so anything we say about the origin side does not generalise to other regions.

The test period is not quite like the training period. Roughly 13,690 test rows contain product categories that did not exist during training, because DataCo's catalogue expanded in 2017. Those rows are encoded as "none of the known categories". Monthly volume in the test window is also about 40% of the earlier level, so the test metrics are a little noisier than the row count suggests.

The models are statistically tied. We report Random Forest as our final model, but the top three are within normal run-to-run variation of each other. A different random seed could reorder them. We would not defend the claim that Random Forest is genuinely the best of the three.

Threshold choice is coarse. Because predicted probabilities cluster rather than spread smoothly, we cannot fine-tune the precision/recall trade-off. We are choosing between a few fixed operating points, and the gap between them is large.

We did not use the clickstream data. We excluded it for a defensible reason, it cannot be joined to orders, but we have not proven it holds nothing useful. A different framing of the problem might find a use for it.

11. Decision log

The major decisions, what else we considered, and what made us choose.

#	Decision	Alternative considered	Why we chose it	Where to find it
1	Read the file as Latin-1	let pandas guess the encoding	UTF-8 corrupts Spanish place names and splits one category into two	Notebook 1, section 1
2	Drop 4 leakage columns	keep them for higher accuracy	the rule "real days > scheduled days" reproduces the label on 97.55% of rows	Notebook 1, section 4
3	Also drop Order Status	keep it (it was not in our original plan)	it is a state that changes after the order is placed, so it is not available at prediction time	Notebook 1, section 4
4	Remove 7,754 cancelled orders	keep them and label them 0	they never shipped, so "late" is undefined; labelling them 0 would teach the model that cancelled means on time	Notebook 1, section 5
5	Chronological split	random train/test split	one order spans up to 5 rows sharing a label, and the real task is forward in time	Notebook 1, section 8
6	Keep outliers	remove or cap them	they are genuine high-value orders; removing them would bias the model against the shipments that matter most	Notebook 1, section 9.4
7	No resampling for imbalance	SMOTE or class weights	1.35:1 is mild, and resampling would	Notebook 1, section 9.1

#	Decision	Alternative considered	Why we chose it	Where to find it
			distort the probabilities our threshold analysis depends on	
8	Add the is_afternoon feature	use the raw hour only	the late rate is a step at noon, not a trend; the flag moved validation ROC-AUC from 0.742 to 0.747	Notebook 1, section 6
9	Exclude order_year	include all calendar features	train is 2015-16 and test is 2017-18, so the year is always out of range at prediction time	Notebook 1, section 6
10	Keep all 132 features	prune to the top 25	the spread across feature sets is smaller than the standard error, and trees select their own features	Notebook 1, section 11
11	Reject PCA	apply it to reduce dimensions	at 25 inputs, feature selection beat PCA by 0.020 ROC-AUC and stayed interpretable	Notebook 1, section 11.4
12	Logistic Regression as baseline	Decision Tree or Naive Bayes	interpretable and fair here, since there is no shipping mode by market interaction to disadvantage a linear model	Notebook 2, section 7
13	Reject KNN and Naive Bayes	include them as alternatives	distance is meaningless across 115 binary columns; one-hot columns break the independence	Notebook 2, section 2

#	Decision	Alternative considered	Why we chose it	Where to find it
			assumption	
14	Custom order-aware CV splitter	plain TimeSeriesSplit	the plain version cut through 2 orders, leaking a label across a fold boundary	Notebook 2, section 3
15	Report the models as tied	name Random Forest the winner	the gap between the top three (0.0022) is smaller than the fold-to-fold variation (0.0030)	Notebook 2, section 12
16	Do not optimise F1 for the threshold	take the maximum-F1 cut-off	maximum F1 flags 97% of orders, which is the "always late" model in disguise	Notebook 2, section 13
17	Tune only Random Forest	tune all of the top models	the gain was inside normal variation, so tuning others could not reorder the ranking	Notebook 2, section 12b

12. Reproducibility

Dataset fingerprint

So a marker can confirm we used the exact file supplied:

File	Bytes	SHA-256
DataCoSupplyChainDataset.csv	95,910,149	fa6d022ed437155e1a2f0378710602848703c8a7f203f7ff5d77805bf8480aa6
DescriptionDataCoSupplyChain.csv	3,444	9828e34669bd6d77e3b4463364cc44a5d52446b5e246fc258758cfe592566c4b

Source: DataCo Smart Supply Chain dataset, as linked in the assignment descriptor for group code 5 (Logistics & Supply Chain).

Environment

Package	Version
Python	3.13.14
pandas	3.0.6
numpy	2.5.3
scikit-learn	1.9.1
scipy	1.18.1
matplotlib	3.11.2
joblib	1.6.0
xgboost	3.4.1

Random seed is 42 throughout, set once in section 0 of each notebook and passed to every split, model and search.

How to re-run

1. Put `DataCoSupplyChainDataset.csv` and `DescriptionDataCoSupplyChain.csv` in the same folder as the notebooks.
2. Run `01_Preprocessing_FINAL.ipynb` top to bottom (about 4 minutes). It writes `artifacts/` and `figures/`.
3. Run `02_Model_Training.ipynb` top to bottom (about 17 minutes, including the tuning search).

Both notebooks are submitted with all outputs saved, so the figures and tables in this report can be checked against them without re-running anything.

Files submitted

File	What it is
`01_Preprocessing_FINAL.ipynb`	EDA and preprocessing, 63 code cells, 31 figures
`02_Model_Training.ipynb`	models, tuning and evaluation, 31 code cells, 9 figures
`artifacts/column_decision_log.csv`	all 35 dropped columns with reasons
`artifacts/processed_data_dictionary.csv`	every column in the processed dataset and its role
`artifacts/model_comparison.csv`	all 8 models and 6 metrics each
`artifacts/manifest.json`	split dates, feature lists, every recorded decision

File	What it is
artifacts/processed_dataset.csv	the cleaned dataset with a split column
figures/	40 charts

13. AI-use declaration

We used an AI assistant (Claude) during this assignment. We are declaring its use openly because the module rewards honesty here.

What we used it for:

- Writing and debugging notebook code, including the preprocessing pipeline, the plotting code and the model comparison loop.
- Suggesting checks we had not thought of, for example, verifying that the cross-validation splitter was not cutting through the middle of an order, and testing whether maximising F1 produces a sensible threshold.
- Drafting the structure and wording of this report from the results already produced by our notebooks.

What we did ourselves:

- Chose the business problem, the primary and secondary lenses, and the task definition.
- Ran both notebooks end to end on our own machines and checked every table and chart before accepting it, rather than trusting the code we were given. Decided which recommendations to put to DataCo and how strongly to word them, including the decision not to claim the tuned model was better than the untuned one.

How we checked the AI's work:

- Every notebook was run end to end on our own machines and the outputs saved. No result in this report was taken on trust.
- Cross-checked every number quoted in this report against the files the notebooks write directly (artifacts/model_comparison.csv, artifacts/model_summary.json and artifacts/final_test_results.csv), so no figure here was typed in by hand.

Errors the AI made that we caught. We list these because finding them is part of how we checked the work, and because a declaration that claims there were none is not believable:

- It recommended TimeSeriesSplit for cross-validation without noticing that the exported rows were not in date order. TimeSeriesSplit assumes row order equals time order, so every fold would have been invalid. We re-exported the splits sorted by order date.
- It said unseen test categories are routed to an infrequent bucket. They are not: the rare-level threshold never fires on this data because every training level already has at least 20 rows, so unseen levels are encoded as all zeros instead.
- It wrote that Shipping Mode carried an order of magnitude more mutual information than any other feature. Latitude actually scored 0.119 against Shipping Mode's 0.146, which is not an order

of magnitude. The wording was corrected and the Latitude result was then shown to be an estimator artefact.

- It stated that Product Price and Order Item Total were not redundant because their correlation was below 1.0. We then found the exact identity Order Item Total = Product Price x Quantity - Discount, which holds on 100% of rows. Correlation had missed it because the relationship is multiplicative.
- It predicted the test score would fall below the validation score because of unseen product categories. The test score actually rose by 0.0103. We corrected the claim and recorded that a rise only means the test window was slightly easier.
- A chart title claimed that three selection criteria agreed on the most important feature. They did not: mutual information disagreed with the other two, and it was the one that was wrong. The title and the surrounding text were rewritten.
- After tuning, the recommended decision threshold of 0.40 was left hard-coded in the final evaluation even though the notebook had recalculated it as 0.38 for the tuned model. We spotted the inconsistency when the two sections disagreed.
- Several plots had layout faults that only showed on inspection: a subtitle printed on top of its title, a box plot where one shared axis squashed every small-scale variable into a line, and a workflow diagram whose feedback arrow was drawn behind the boxes. Each was found by looking at the rendered image rather than trusting the code.

14. Member contributions

Member	IT Number	Focus area	What they did
Bolonne B.R.M.	IT24102050	EDA + Preprocessing	EDA and dataset cleaning, dropping leakage and non-predictive columns, filtering cancelled orders, encoding
Kodagoda D.S.	IT24101613	EDA + baseline model training	Training the baseline Logistic Regression model, Decision Tree, Random Forest, XGBoost and neural network models
Jayasekara C.D.	IT24103494	EDA + Advanced models and tuning	Hyperparameter tuning on the strongest candidate
Liyanage J.L.K.L.	IT24101927	EDA + Evaluation and recommendation	Metric comparison across all models, cross-validation, the

Member	IT Number	Focus area	What they did
			final recommendation with limitations

15. Appendix

Name	Description	Link
GitHub repo link	GitHub repo containing all notebooks and project materials	https://github.com/Rosheni-a11y/IT3091_ML_Assignment.git
Demo link	Unlisted YouTube link containing a demonstration of the project	https://youtu.be/IfhiaASmYhE
Deployed website link	An interactive Streamlit demo website featuring model performance, key findings, and final recommendations alongside live scoring and custom order simulation.	https://it3091mlassignment-a5pssekjqfbdbprx7e7chu.streamlit.app/